

Script oral — Une vie de fourmi (5 minutes)

Équipe : Romain · Lisa · Yannis

Durée visée : 4 min 40, soit environ 20 secondes de marge sur 5 minutes

Qui	Slides	Temps
Romain	1 et 2 : le problème, les données	1 min 20
Lisa	3 et début de la 4 : l'algorithme, l'animation	1 min 30
Yannis	fin de la 4, 5 et 6 : trafic cumulé, validation, conclusion	1 min 20

Règle d'or : on ne lit pas l'écran. Une idée par slide, on montre du doigt ce dont on parle, et on passe la parole par une phrase de transition.

Slide 1 — Le problème — Romain — 40 s

Bonjour, nous sommes Romain, Lisa et Yannis. Le soir, toute la colonie attend dans le vestibule, Sv, et doit rejoindre le dortoir, Sd, le plus vite possible.

Trois règles : les fourmis bougent en même temps ; chaque salle a une capacité limitée, une seule fourmi par défaut ; et on veut le minimum d'étapes, c'est-à-dire que la dernière fourmi arrive le plus tôt possible.

Si toutes les fourmis prennent le plus court chemin, ça bouchonne. Ce n'est pas un problème de chemin : c'est un problème de trafic.

Geste : montrer Sv à gauche de l'image, puis Sd à droite, puis les salles cerclées de rouge (pleines).

Slide 2 — Les données — Romain — 40 s

Voici un vrai fichier officiel : 50 fourmis, 14 salles, 20 tunnels. « S1 accolade 8 » veut dire que S1 accueille 8 fourmis à la fois ; sans accolade, c'est une seule.

On le transforme en graphe : une salle devient un sommet, un tunnel devient une arête. Les tunnels n'ont pas de limite, ce sont les salles qui limitent.

Notre programme lit les neuf fichiers officiels tels qu'ils sont fournis, espaces compris. Lisa va vous expliquer comment on trouve le minimum.

Geste : montrer le fichier à gauche, suivre la flèche, puis montrer S1 (capacité 8) sur le graphe.

Slide 3 — Comment l'algorithme réfléchit — Lisa — 55 s

D'abord un BFS, un parcours en largeur : il donne la distance minimale d entre Sv et Sd. Aucune fourmi ne peut arriver avant d étapes.

Ensuite, on ne cherche pas un chemin : on pose une question. En T étapes, est-ce que les 50 fourmis peuvent passer ?

Pour y répondre, on déplie le graphe dans le temps : une copie de chaque salle à chaque instant. Une flèche horizontale, c'est attendre ; une flèche en biais, c'est traverser un tunnel. La capacité d'une salle devient la capacité de sa copie.

Un algorithme de flot maximal compte combien de fourmis peuvent arriver. Sur le graphique : 4 fourmis en 5 étapes, 12 en 6... et 50 en 11. Onze, c'est le premier T qui marche, donc c'est le minimum.

Geste : suivre une flèche orange sur le graphe déplié, puis montrer la barre orange du graphique.

Slide 4 — Voir les fourmis circuler — Lisa puis Yannis — 55 s

Lisa (35 s). Voici le résultat, étape par étape. Tout le graphe reste affiché, même les tunnels jamais utilisés, en gris. En orange, les fourmis qui se déplacent pendant l'étape, avec leur nombre. Dans chaque salle : occupation sur capacité ; un cercle rouge, c'est une salle pleine. En haut, le compteur d'arrivées. Yannis va vous montrer le bilan.

Yannis (20 s). À droite, le trafic cumulé : plus un tunnel est épais, plus il a servi. On voit que le flot se répartit sur toutes les branches, et que le goulot limite à 8 arrivées par étape.

Geste : laisser tourner le GIF une boucle ; montrer une flèche orange et un cercle rouge.

Slide 5 — Validation sur les vraies données — Yannis — 45 s

Le programme n'a pas été écrit pour un seul exemple : les neuf fichiers officiels sont résolus, de 2 à 100 fourmis, sans aucun réglage.

Chaque solution est vérifiée automatiquement : capacité de chaque salle à chaque instant, tunnels existants, toutes les fourmis dans Sd.

Et pour prouver le minimum, on a comparé notre résultat à une recherche exhaustive de toutes les configurations sur 400 petites fourmilières aléatoires : même nombre d'étapes à chaque fois.

Geste : montrer la ligne à 100 fourmis, puis la carte « 400 / 400 ».

Slide 6 — Conclusion — Yannis — 25 s

En résumé : c'est correct, c'est optimal, et c'est visuel.

Le plus court chemin optimise une fourmi. Notre solution optimise toute la colonie.

Merci, nous sommes prêts pour vos questions.

Questions probables : réponses courtes (chacun doit les connaître)

Pourquoi un graphe ?

Salles et tunnels, c'est exactement un graphe : une salle est un sommet, un tunnel une arête. Ça nous donne gratuitement les voisins, les distances et les algorithmes de flot.

Pourquoi ne pas simplement utiliser BFS ?

On utilise justement BFS pour connaître la distance minimale entre le vestibule et le dortoir. Mais BFS trouve un chemin pour un trajet individuel. Notre problème demande de coordonner plusieurs fourmis en parallèle avec des capacités. Pour cela, on transforme le problème en réseau de flot temporel. (Annexe « Pourquoi pas simplement BFS ou Dijkstra ? » : avec 3 fourmis, le plus court chemin seul prend 4 étapes, notre solution 3.)

Où est le BFS exactement ?

Dans `ants.bfs_distances`, pour la borne basse `d`. Et Edmonds-Karp, notre max-flow, utilise lui aussi un BFS pour trouver ses chemins augmentants dans le graphe résiduel.

Pourquoi pas Dijkstra ?

Tous nos tunnels représentent exactement une étape, donc ils ont tous le même coût. Dijkstra donnerait la même distance que BFS en étant inutilement plus général. Et comme BFS, il ne gère pas à lui seul le trafic multi-fourmis.

Pourquoi pas DFS ?

DFS permet d'explorer le graphe, mais il ne garantit pas le plus court chemin et ne résout pas les contraintes de trafic. Nous n'avons donc aucune raison de l'utiliser ici.

Et Floyd-Warshall ?

Il calcule toutes les distances entre toutes les paires de salles ; on n'a besoin que des distances depuis `Sv` et vers `Sd`.

Qu'est-ce qu'un max-flow ?

C'est la quantité maximale qu'on peut faire passer d'une source à un puits sans dépasser la capacité de chaque passage, comme de l'eau dans des tuyaux. Ici, une unité de flot, c'est une fourmi.

Pourquoi le premier T faisable est-il optimal ?

On teste $T = d$, puis $d + 1$, $d + 2...$ dans l'ordre. Avant d , c'est impossible. Pour chaque T , le graphe déplié contient tous les déplacements autorisés : si le max-flow est inférieur à F , aucun planning en T étapes n'existe. Le premier T qui marche est donc le plus petit possible.

Comment les capacités sont-elles représentées ?

Chaque copie de salle est coupée en deux : une entrée et une sortie, reliées par un arc de capacité X . Au plus X fourmis peuvent donc s'y trouver au même instant. C'est le « node splitting ».

Et « entrer pendant que l'occupante sort » ?

On ne compte l'occupation qu'à la fin de chaque étape : une fourmi peut entrer dans une salle au moment où l'autre en sort. C'est exactement la règle du sujet.

Pourquoi toutes les fourmilières fonctionnent-elles maintenant ?

Le lecteur de fichier accepte le vrai format officiel : espaces dans les accolades, « $f=50$ », lignes vides, fins de ligne Windows. Les neuf fichiers sont testés automatiquement à chaque modification.

Le tunnel direct de fourmiliere_deux ?

`Sd-Sv` existe : les tunnels n'ont pas de limite et `Sd` non plus, donc les 5 fourmis arrivent toutes en 1 étape.

À quoi sert le min-cost flow ?

Une fois T fixé, plusieurs plannings sont optimaux. Le min-cost choisit le plus lisible : les arrivées les plus précoces, puis le moins de mouvements, puis aucun pas en arrière inutile. Il ne change jamais T .

C'est rapide ?

Moins de 0,3 seconde pour le plus gros fichier (100 fourmis). On a gardé la recherche linéaire de T : la recherche binaire serait correcte, mais elle est plus lente ici.